

Does Tool Schema Design Bound the Root-Cause Discovery Rate of a Tool-Calling LLM Agent on Post-Silicon ATE Data?

Venkata Naga Mayank Konduri¹

¹Dept. of Electrical and Computer Engineering, The University of Texas at Austin, Austin, TX 78712 USA

Index Terms—Automatic test equipment (ATE), gate dielectric defects, large language models, post-silicon validation, root-cause analysis, standard test data format (STDF), tool-calling agents, yield analysis, yield excursion.

I. INTRODUCTION

Modern semiconductor companies can design advanced chips through a fabless manufacturing model. Wafers are produced by external foundries, while assembly and test are performed by partner facilities, often overseas [1]. The test equipment returns data rather than physical silicon: automatic test equipment (ATE) records measurements in Standard Test Data Format (STDF), which is converted and stored in a central yield database [2], [3], [4], [5].

A production lot is a group of wafers processed under the same manufacturing conditions. When a lot's yield falls outside its normal range, an automated rule flags it as a yield excursion for engineering review [6]. By the time a yield excursion reaches a product engineer, the root-cause is already in the ATE datalog returned from the ATMP partner; what remains is knowing how to uncover it. A pattern may remain invisible until the data is sliced along the right dimensions and narrowed to the right conditions, making the decision of where to look next central to diagnosis: a decision traditionally left to engineers.

A. Motivation

As semiconductor test moves toward agentic analysis, large language models (LLMs) introduce the possibility of automating this decision loop. A tool-calling LLM can form a hypothesis, select an analysis function, inspect its result, and use that result to determine its next action [9]. In this setting, the tools perform the computation, but the LLM provides the adaptive reasoning that previously came from the engineer. The central question is therefore not whether an LLM can analyze test data, but *what determines whether an agent can make that decision reliably, and does the design of its tools limit its diagnostic accuracy?*

To investigate this, analysis tools available to a tool-calling agent are varied while holding the model, prompt, and data fixed, measuring how tool design affects which failure dimensions the agent recovers and how precisely it locates them.

II. BACKGROUND AND SCOPE

In AMD's production flow, product development and test engineering remain with AMD while fabrication and assembly/test are performed through external manufacturing partners

[1]. AMD identifies TSMC and GlobalFoundries as wafer-fabrication partners, while Tongfu ATMP joint ventures and SPIL are among its assembly and test providers [1]. These partners perform wafer sort, assembly, and production test using automatic test equipment (ATE). ATE vendors such as Teradyne and Advantest provide the equipment used by ATMP partners such as Tongfu to execute semiconductor test programs and collect test results [2], [3].

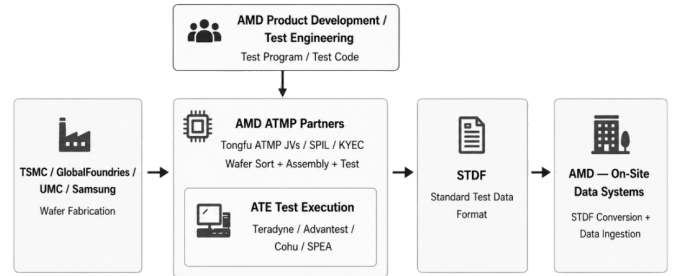


Fig. 1. Simplified industry post-silicon test-data flow. AMD develops the test program, which is executed by ATE at an ATMP partner. The resulting STDF datalog is subsequently ingested into AMD's on-site data systems [1], [2].

The ATE produces test measurements and pass/fail results as an STDF datalog, which is stored as a stream of binary records containing lot, wafer, site, part, and individual test information [2], [4]. Its standardized structure allows data from different test systems to be centrally processed and analyzed [5]. This test data reaches AMD's on-site data systems and forms the basis of the engineering analysis considered in this work.

This boundary defines the scope of the study. The experiment does not reproduce wafer fabrication, physical ATE execution, or AMD's proprietary data-ingestion infrastructure. Instead, the STDF data are simulated after conversion into a structured table for analysis. The study then evaluates how an LLM agent investigates this data using different analysis-tool schemas. Test-program modification and on-tester adaptive testing are also excluded from the study [11], [12].

III. DATASET PREPARATION

Two synthetic post-silicon test datasets were generated and stored as CSV files to emulate structured ATE data after STDF conversion [2], [3]. Each file contains five lots, with five wafers per lot and 150 die per wafer, for a total of 3,750 parts. Each part is evaluated at three temperature insertions (0°C, 25°C, and 85°C) and three supply levels ($V_{DD} = 0.75\text{--}0.85\text{ V}$)

across ten parametric tests, producing 112,500 test-execution records per file and approximately 11.8 MB of data. Die are assigned to eight parallel test sites based on wafer position to approximate multisite ATE testing [2].

TABLE I
SYNTHETIC DATASET STRUCTURE

Field	Description
LOT_ID	Lot identifier
WAFER_ID	Wafer identifier
PART_ID	Die / part identifier
X_COORD, Y_COORD	Die position
SITE_NUM	ATE test site (1–8)
TEMP_C	Test temperature
TEST_NUM, TEST_TXT	Test identifier / name
RESULT	Measured value
LO_LIMIT, HI_LIMIT	Specification limits
UNITS	Measurement units
HARD_BIN, SOFT_BIN	Test outcome bins
TIMESTAMP	Test execution timestamp

Fig. 2. Synthetic dataset structure. The table summarizes the STDF-derived fields used to represent part identity, test conditions, measurements, specification limits, bin outcomes, and test execution information.

As shown in Fig. 2, the columns preserve the information needed for post-silicon investigation. LOT_ID, WAFER_ID, and PART_ID identify the tested part and its manufacturing context, while X_COORD and Y_COORD preserve its spatial location. SITE_NUM and TEMP_C capture the test conditions, while TEST_NUM, TEST_TXT, and RESULT describe the measurement. Specification limits, bin results, units, and the execution timestamp provide the information needed to determine and characterize failures.

A. Failure Injection

Two datasets were generated using the same random seed and differ only by the injected condition. The first, `clean.csv`, serves as the control. Its measurements follow normal distributions with limits set to represent nominal device behavior, resulting in 95.8% part-level yield. Random failures are retained to verify that the agent can distinguish normal variation from a true failure pattern.

1) Physical Failure Scenario

For `defect_4d.csv`, the injected condition models a localized process issue during the formation of the HfO_2 gate insulator in a transistor. The insulator is formed using atomic layer deposition (ALD), which alternates between a hafnium precursor pulse and an ozone (O_3) pulse. The precursor deposits hafnium on the wafer, while ozone removes the precursor’s organic groups and completes the oxidation. The scenario assumes a partial ozone valve blockage that reduces the ozone dose by approximately 10% in one chamber region while leaving the hafnium dose unchanged.

2) Transistor-Level Impact

The reduced ozone dose leaves oxygen vacancies in the HfO_2 layer. These defects introduce trapped charge and scattering near the gate–channel interface, reducing the transistor’s effective drive current. In simple terms, the defect acts like a rough section of road: the gate provides the force that moves charge, while the defects make that movement less efficient.

The resulting reduction in drive current can be expressed as

$$I_D \propto \mu_{\text{eff}} C_{\text{ox}} (V_{DD} - V_{\text{th}})^{1.3}, \quad (1)$$

where μ_{eff} is the effective carrier mobility and V_{th} is the threshold voltage. The defects increase V_{th} and reduce μ_{eff} , causing I_D to decrease. Consequently, the transistor requires a higher minimum voltage to operate and switches more slowly. Since switching delay is approximately

$$t_{pd} \approx \frac{C_L V_{DD}}{I_D}, \quad (2)$$

a lower drive current produces a longer delay and therefore a lower maximum operating frequency.

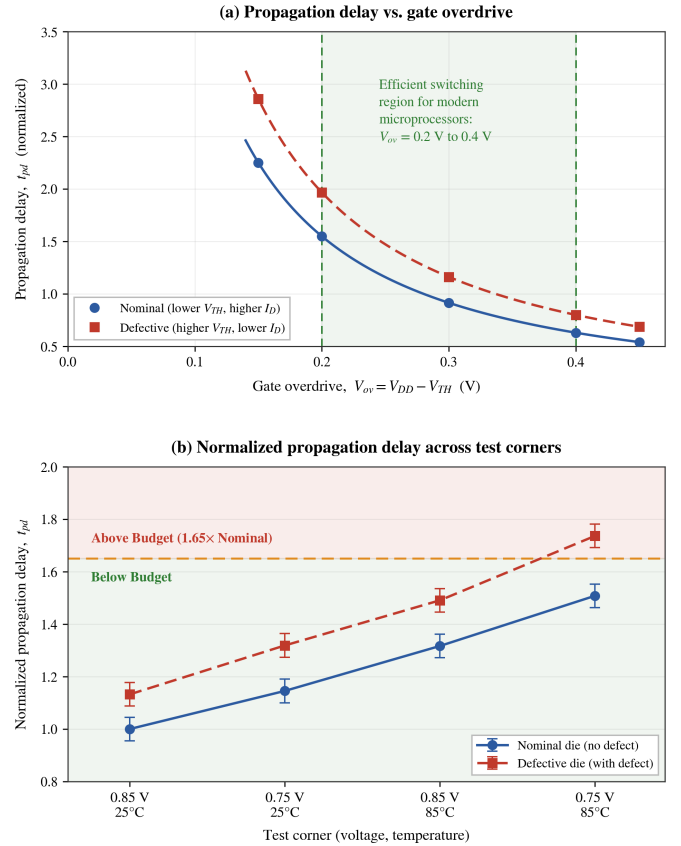


Fig. 3. Illustration of the transistor-level impact of reduced gate overdrive and temperature on propagation delay. The defective device exhibits greater delay and exceeds the timing budget at the hot, low-voltage corner.

As illustrated in Fig. 3, the effect becomes more visible at high temperature and low supply voltage. Temperature reduces carrier mobility [15], while lower V_{DD} reduces the voltage margin available above V_{th} [13]. Thus, either condition can

reduce the available timing margin, but their combination can cause defective die to fail while otherwise healthy die continue to pass. In the synthetic dataset, this behavior is represented by increased `vmin_core` and reduced `core_fmax` at the 85°C insertion.

3) Injected Conditions and Affected Tests

TABLE II
INJECTED FAILURE CONDITIONS AND AFFECTED TESTS

Dimension	Injected condition
Lot	L46, L47
Wafer region	Upper-left perimeter sector
Spatial range	$r > 0.55$, angle 100°–195°
Temperature	85°C
Supply voltage	$V_{DD} = 0.75$ V
Affected test	Effect
<code>core_fmax</code>	Decreases due to slower switching
<code>vmin_core</code>	Increases due to reduced drive current
<code>pll_lock</code>	Degrades from increased timing delay
<code>sram_bist</code>	Degrades from increased timing delay
Other six tests	Relatively unaffected

Fig. 4. Tabular summary of injected failure conditions and affected tests. The condition is localized to two lots, a wafer-perimeter sector, the 85°C insertion, and the 0.75 V supply level, affecting four of ten tests.

According to Fig. 4, the injected condition is defined by the intersection of four dimensions: two lots, a localized wafer region, the 85°C insertion, and the 0.75 V supply level. The issue emerges only when all four conditions coincide, as the affected die remain near their operating limits under the other conditions but fail when these conditions combine. Together with the four affected tests, this produces only 339 affected rows, or 0.28% of the full dataset, but these rows account for 199 of the 3,750 die, or 5.2% of parts. This difference occurs because a die is considered failed if even one of its 30 measurements fails, allowing a small number of affected measurements to cause failures across many die.

IV. ANALYSIS TOOLS

A. Lot Screening and Excursion Detection

At this stage of the production flow, the ATE datalogs generated at the ATMP partner are transferred back for analysis [1]. Test results are recorded in STDF, a binary record stream containing lot, wafer, site, part, and individual test information [2], [3]. Because STDF is designed for recording test data rather than analyzing it, the binary stream is parsed into a structured CSV table before investigation begins [4], [5]. The analysis in this work uses this converted datalog as its input.

The first step is an automated triage gate implemented as the `triage.sh` script, which determines whether an incoming lot needs investigation. This *gate* combines individual measurements into die-level yield, compares the yield with a statistical limit based on prior production history, and produces a result. AEC-Q002 suggests this approach for identifying wafers or lots with unusually low yield, using limits based on previous lots [6]. The gate also reports yield for each lot

and the number of failures by test. It does not search for the root cause; it only determines whether an excursion is present.

The gate issues no production decision. If a lot passes, the agent is not invoked; the gate report is retained for human review and the engineer may release the lot. If a lot fails, the report becomes the initial context for the analysis agent, which continues the root-cause investigation using the tools described below. Disposition remains a human judgement in either case [6].

For the experiment described in this work, this rule is deliberately overridden. Regardless of whether the gate flags an excursion, the agent is also invoked on the control dataset. The control dataset passes the gate and would therefore never reach an agent in production. Running this case measures how often a root cause is reported where none exists, which is otherwise unobservable.

B. Query Functions

When a lot is flagged, the agent is invoked with the converted datalog and the gate report as its starting context. As defined in `analyze.py`, the agent can ask up to six types of questions, depending on the tool configuration, with each question returning a compact numerical summary. The functions do not rank, flag, or interpret results; all reasoning remains with the agent. The agent therefore controls both the sequence and scope of its investigation, rather than following a fixed analysis procedure. This distinction is necessary for the tool-schema comparison in Section V-B.

TABLE III
QUERY FUNCTIONS AND ANALYSIS DIMENSIONS.

Function	What the Function Reveals
<code>get_overall_yield</code>	Overall yield across the entire dataset
<code>get_failures_by_test</code>	Failure count and rate for each test
<code>get_yield_summary</code>	Yield summarized by lot, wafer, and site
<code>get_fail_rate_by_condition</code>	Failure rate across temperature and voltage
<code>get_yield_by_region</code>	Yield across wafer regions (radial band × octant)
<code>get_distribution_stats</code>	Distribution statistics of a measured parameter

Fig. 5. Summary of the query functions available to the agent and the data dimensions they allow it to examine during root-cause analysis.

As shown in Fig. 5, the six question types provide the agent with different views of the data [16], [17]. Each function also accepts optional filters that allow the agent to focus the result on a specific subset of the data. This allows the agent to carry a finding from one query into the next and examine a progressively smaller population. For example, after identifying a suspect lot, the agent can restrict subsequent queries to that lot and then examine temperature, voltage, test, or wafer region. Without filtering, each query provides only a broad view of the data and prevents the agent from following findings into more specific subsets. The presence or absence of this filtering capability is ultimately a key variable in Section V-B.

C. Agent Tool Interface and Schema Configurations

The agent does not directly access the datalog or call the underlying functions. Instead, a `tools.py` interface exposes selected query functions to the agent through JSON schemas, which specify the available functions, their arguments, and the information they return. This interface enables an experiment across four configurations that vary two parts of the tool schema: *coverage*, which determines how many types of questions the agent can ask, and *composition*, which determines whether the agent can add filters to focus a query on specific conditions. The four configurations form a 2×2 design varying the number of query functions and whether filters are available (Figure 6).

TABLE IV
TOOL CONFIGURATIONS USED IN THE EXPERIMENT.

	No Filters	Filters
2 Functions	minimal	minimal_f
6 Functions	reduced	full

Fig. 6. Tool configurations used in the experiment. The four configurations vary the number of exposed query functions and whether filters are available.

The model, dataset, and prompt remain the same across all configurations, making this comparison a direct test of how tool schema design, through the breadth of available questions (coverage) and the ability to combine conditions (composition), affects the agent’s analysis.

D. Agent Architecture

When the triage gate flags an excursion, the harness invokes `agent.run()` with three arguments: the converted datalog, the tool configuration under test, and the gate report. The agent constructs an opening message from the report and enters an iterative reason–act loop [9].

At each iteration, the agent sends the system prompt, complete conversation history, and JSON schemas for the active configuration to the model [10]. As shown in Fig. 7, this information is sent from `agent.py` to the language model (Step 1). The model returns either a structured tool request or a final report (Step 2). For a tool request, `agent.py` passes it to `tools.py` (Step 3), which verifies and calls the corresponding function in `analyze.py` (Step 4). The data returned from `analyze.py` are passed back through `tools.py` (Step 5) and returned to `agent.py` as a compact summary (Step 6). The result is appended to the conversation history and retransmitted to the model for the next iteration. Because the model retains no state between calls, the full history is retransmitted at each iteration, keeping previous results available. The loop ends when the model returns a report or after sixteen iterations.

The division of labor is central to this work. All numerical operations are performed by `analyze.py`, which filters rows, groups the remaining data, and returns failure rates. The model instead determines the sequence of analysis decisions: which dimension to examine, whether an observed difference warrants further investigation, which condition to hold fixed while varying another, and when the available evidence is sufficient to conclude. This is the decision process under study.

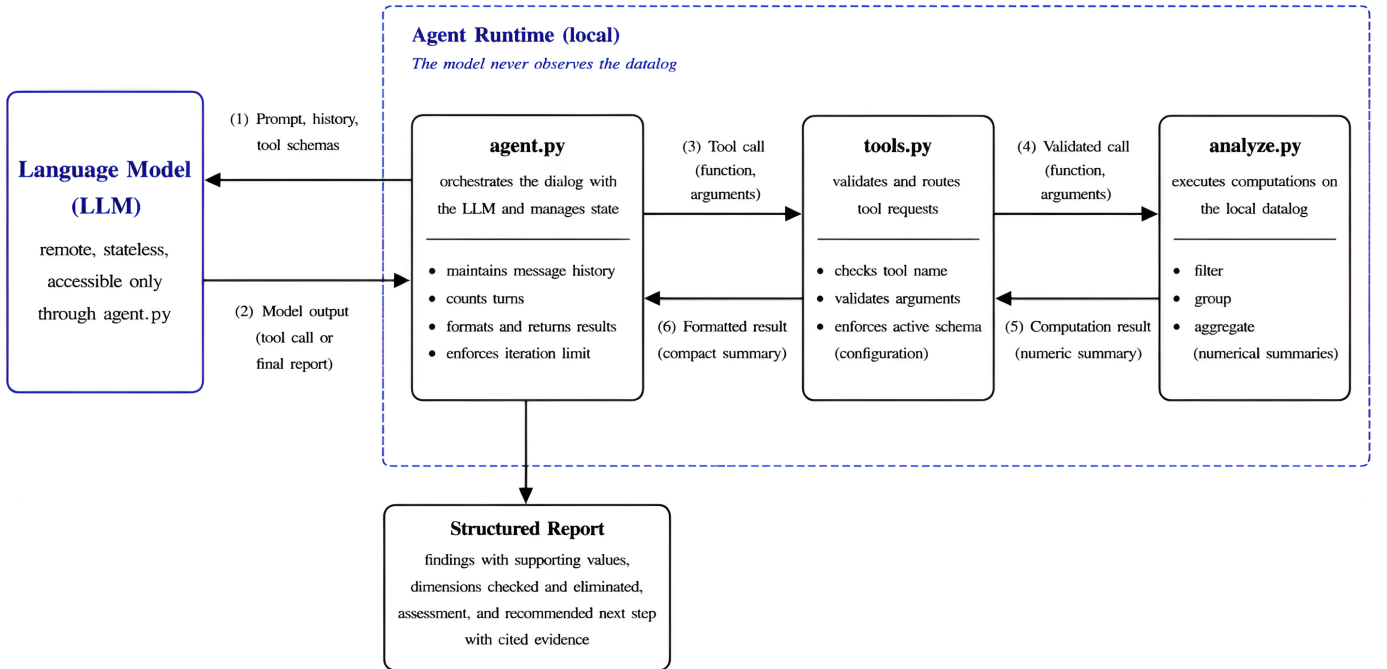


Fig. 7. Agent architecture and data flow. The language model (LLM) is accessible only through `agent.py`. At each turn, the agent sends the prompt, history, and active tool schemas to the LLM and receives either a tool call or the final report. Tool calls are validated by `tools.py` and executed by `analyze.py` on the local datalog. Results are summarized, returned to `agent.py`, appended to the history, and retransmitted to the LLM. The loop repeats for up to sixteen iterations or until a final report is produced.

A failure rate grouped by lot is computationally simple; the challenge is deciding which aggregation to request based on the previous result.

This division motivates the experimental variable. The model determines which analyses to perform, while the tool schema determines which of those analyses are available to it. Therefore, the schema places a direct limit on what the agent can investigate. A dimension that is not supported by any exposed function cannot be examined, and conditions that cannot be combined through filters cannot be tested together. Section V-B varies these two properties directly.

Each run ends with a structured report containing the findings and the values supporting them, the dimensions checked and eliminated, an assessment, and a recommended next step. The report is rendered as a PDF for engineering review, with an appendix listing every query the agent issued so that each finding can be traced to the data that produced it. The system prompt requires each claim to cite the retrieved values, not present correlation as physical root cause. These constraints expose the limits of the available data. The agent sees only the datalog and has no access to process or equipment records, so it can identify where a failure occurs in the data but cannot determine its physical cause. This limitation is reflected in the final report.

V. EXPERIMENTAL SETUP

A. Objective

The experiment tests whether tool-schema design, rather than the reasoning capability of the model, bounds what an agent can discover in post-silicon test data. Two properties of the schema are varied independently.

Coverage is the number of query functions exposed and therefore the set of analysis dimensions the agent can observe. A dimension that no exposed function can group by cannot be examined, regardless of how the agent reasons.

Composition is whether filters are exposed and therefore whether the agent can use the result of one query to restrict a subsequent query to a specific set of conditions. Without composition, each query examines its grouping dimension independently.

The distinction matters because the injected condition affects only 0.30% of measurements and is defined by the intersection of four dimensions. Examining each dimension separately raises the failure rate only to 0.66%–1.47%, compared with a 0.14% baseline. Thus, each dimension shows a signal, but none is conclusive alone. Combining all four conditions yields 72.0%, which an agent without composition cannot directly verify. Therefore, two claims follow:

- (1) **Coverage bounds which dimensions are reported.**
Configurations exposing fewer grouping functions report fewer of the four injected dimensions.
- (2) **Composition bounds how deep the agent can search.**
Configurations without filters may report the correct dimensions but cannot quantify their joint effect.

A second objective concerns behavior in the absence of a defect. The control dataset contains no injected condition, so

the correct result is that no systematic pattern is present. The agent is also run on this dataset, so the frequency with which it reports dimensions provides a measure of false-positive behavior that would otherwise be unobservable. Across all configurations, the model, system prompt, datasets, iteration limit, and query-function implementations remain unchanged. The only variable is which schemas are exposed and whether they accept filters.

B. Experimental Protocol

Each of the four tool configurations, as shown in Fig. 6, is run five times on each of the two datasets, the injected and control datasets. The 40 runs are launched by a single execution of `run_matrix.py`, giving a total of $4 \times 2 \times 5 = 40$ runs. Repeats are required because the investigation is stochastic: the model makes its analysis decisions from the accumulated conversation, selecting which query to issue based on the results it has already observed. This decision-making is abstracted from the underlying numerical analysis, so identical runs may make different query choices and follow different investigative paths.

Every run finishes with two labelled lines. These are the agent’s answer, and they are what the scoring reads. Only these lines are scored: the report body is written to disk with each run’s transcript and remains available for inspection, but is not part of the automatic evaluation. PDF reports can be rendered individually for runs, if examined in detail.

The first line lists the dimensions the agent blames. It may name any of `LOT_ID`, `WAFER_ID`, `SITE_NUM`, `TEMP_C`, `VDD`, and `SPATIAL`, or `NONE` if it found no pattern. The second line names the particular values within each dimension it named, for example `LOT_ID=L47,L48; TEMP_C=85`.

The two lines are separate because they test different aspects of the investigation. Saying that die position matters is an example of coverage, since the agent must be able to examine `SPATIAL`. Saying *which* part of the wafer fails is an example of composition, since the agent must isolate the relevant conditions. An agent can therefore have sufficient coverage to identify where to look but insufficient composition to determine the specific condition.

As discussed in Section III, the injected dataset has `LOT_ID`, `SPATIAL`, `TEMP_C`, and `VDD` as the correct dimensions, with the corresponding values of lots L47 and L48, 85°C, 0.75 V, and the NW octant at outer radius. For the control, the correct answer is `NONE`, because nothing was injected.

Each line produced by `run_matrix.py` is then compared against these answers, recording how many entries were correct, incorrect, and missed, as well as whether the set matches exactly. Answering `NONE` and failing to produce the line at all are recorded separately, since the first is an answer and the second is not.

VI. RESULTS

Across the 40 runs, two patterns emerge: coverage was a key factor in determining which dimensions the agent could discover, while composition was a key factor in determining

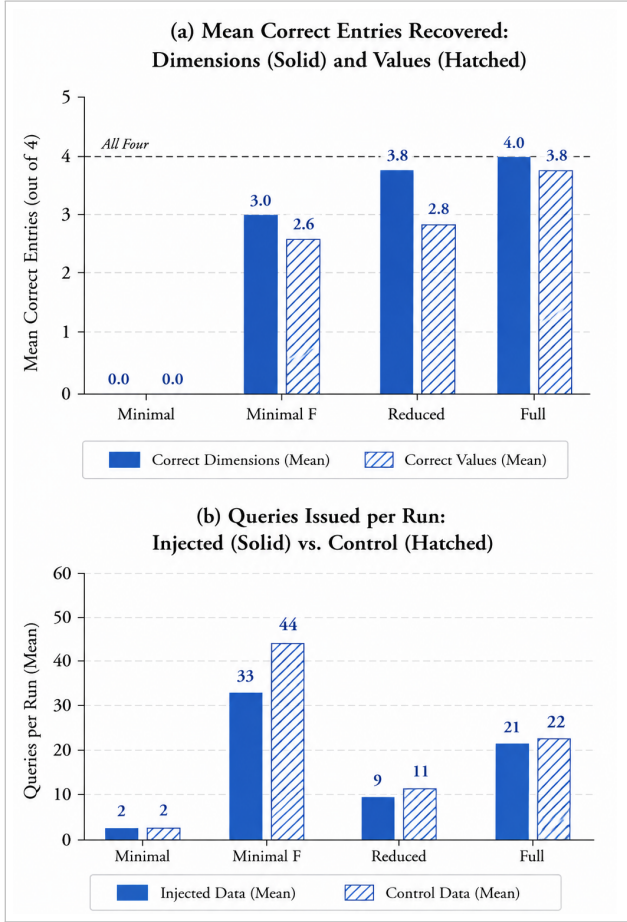


Fig. 8. Results across the four tool configurations. (a) Mean correct dimensions and specific values recovered from the injected dataset, illustrating the effects of tool coverage and composition. (b) Mean queries issued per run for the injected and control datasets.

how deeply it could investigate their relationships. These trends are summarized in Fig. 8.

A. Coverage Bounds Discovery

`minimal` recovered no dimensions in any run and stopped after two queries because no further query was available. This is search exhaustion, not search failure. Although the gate report identified L47 and L48 as held lots, `minimal` still reported `NONE` because the agent can only make claims supported by its retrieved data.

`minimal_f` recovered three of the four dimensions in all five runs, missing `LOT_ID` every time. As shown in Fig. 8(a), it recovered a mean of 3.0 dimensions and 2.6 corresponding values. This limitation is structural: the agent can filter to a specific lot but cannot group results by lot, so it cannot discover which lots are abnormal without already knowing which ones to test. Filters can therefore partly compensate for missing coverage, but at a cost: `minimal_f` issued 33.0 queries per run, compared with 2.0 for `minimal`, as shown in Fig. 8(b).

B. Composition Bounds Search Depth

`reduced` and `full` found nearly the same number of correct dimensions, 3.8 and 4.0, respectively. As shown in

Fig. 8(a), the difference appears when the agent must determine the specific values associated with those dimensions. Here, a correct value means identifying the specific condition responsible for the failures, such as `TEMP_C=85` or the NW octant at outer radius. `reduced` identified 2.8 correct values compared with 3.8 for `full`.

Without filters, `reduced` could see that each dimension was associated with higher failure rates, but could not reliably test whether those conditions occurred together. It therefore treated the dimensions as independent effects in four of five runs. One run inferred the interaction from the marginal results alone, suggesting that coverage can substitute for composition in edge cases, although composition makes this inference much more reliable. This substitution, however, comes at a cost: `full` used 21.0 queries per run versus 9.0 for `reduced`, corresponding to approximately 2.5 $\times$ the token usage. The higher query cost is also visible in Fig. 8(b).

C. False Discovery on the Control

As shown in Fig. 8(b), the control generally required more queries than the injected dataset because the agent continued searching for an explanation even though no systematic condition was present. Although `NONE` was explicitly allowed as a valid answer, the agent’s effort to identify the most plausible cause led it to examine increasingly small patterns in the naturally occurring failures. Here, a *spurious dimension* is a dimension the agent associates with failure even though no condition was injected along that dimension. As a result, 22 of 25 control runs declared at least one spurious dimension, with mean counts of 0.0, 2.6, 2.4, and 2.8 across the four configurations. Importantly, all spurious declarations corresponded to failure rates below 1.0%, suggesting that an engineer reviewing a flagged column would recognize these as relatively minor issues rather than likely systematic causes. Distinguishing such naturally occurring variation from a systematic condition would require additional statistical validation, which is beyond the scope of this research question.

D. Performance Under Full Capability

`full` achieved near-perfect results on the injected dataset, with exact agreement in 4 of 5 runs and only one missed exact match. By combining all six query functions with filters, it could both identify the correct dimensions and investigate their specific values and relationships. This greater capability came at a cost: `full` issued 21.0 queries per run, more than twice the 9.0 used by `reduced`. Overall, the results show that full capability substantially improves the depth and reliability of the agent’s analysis, with only one run failing to match the complete ground truth.

VII. CONCLUSION

The forty-run matrix exists to measure the agent, but it is not how the agent would be used. In production, `run_matrix.py` would not run at all. An engineer would receive a report only when the automated triage gate held a lot, and that report would name the columns where the agent believes the problem lies, give its estimate of what

is happening, and list every query it issued to reach that conclusion. The purpose of the agent is not to produce a finished answer, but to narrow the search and provide evidence for the engineer [18], [19], [20]. The engineer begins with a narrowed population, a numerical basis for the agent’s findings, and a visible trail of what was already checked, and can then reason about the physical cause using the process and equipment records the agent never had.

That framing sets up the question this work asked: when an agent investigates test data, is what it finds limited by the model’s reasoning, or by the questions its tools let it ask?

A. Study Takeaway

Statistical tests on the individual runs show that both coverage and composition significantly affect the agent’s performance. For coverage, a two-sided permutation test comparing the number of correct *dimensions* recovered by `minimal_f` and `full` gave $p = 0.008 < 0.05$. The null and alternative hypotheses were

$$H_0: \mu_{\text{minimal_f}} = \mu_{\text{full}}$$

$$H_1: \mu_{\text{minimal_f}} \neq \mu_{\text{full}}$$

The result indicates a statistically significant improvement with greater coverage. For composition, a two-sided permutation test comparing the number of correct *values* recovered by `reduced` and `full` gave $p = 0.021 < 0.05$. This indicates a statistically significant improvement with greater composition, although the evidence is not as strong as for coverage, as shown in Fig. 8(a). The hypotheses for this comparison were:

$$H_0: \mu_{\text{reduced}} = \mu_{\text{full}}$$

$$H_1: \mu_{\text{reduced}} \neq \mu_{\text{full}}$$

Together, these results provide statistical support for the central finding of this study: tool coverage determines what the agent can discover, while tool composition determines how precisely it can investigate those findings.

The results also show that greater capability is not automatically better. More powerful and more lenient tools allowed the agent to investigate further by using more queries and tokens, but also gave it more opportunities to explain small amounts of naturally occurring variation as a cause. This makes tool design a central part of reliable automated triage: the tools must provide enough coverage and composition to support useful reasoning without encouraging unsupported conclusions.

Taken together, these findings suggest that an effective triage agent should be viewed as an investigation aid rather than an autonomous root-cause authority. Its strongest contribution is narrowing the search and providing evidence that an engineer can evaluate against physical process knowledge.

B. Future Work

A natural next step is to let the agent evaluate the test program itself. Given data from many lots, the agent could identify tests that rarely provide useful information, produce results that are redundant with other tests, or could be moved

earlier to detect failures sooner. This could help reduce test time while maintaining the ability to diagnose failures [11].

However, removing a test can reduce visibility into rare failure modes. Therefore, the agent should produce a ranked list of test-program changes for engineers to review rather than make changes automatically. Any approved change would need to follow the qualification and customer-notification processes required for test-program revisions [7], [8].

Future work could also test whether statistical safeguards reduce the false discoveries observed in the control data, and whether stronger models can overcome some of the limitations imposed by the available tools.

IMPLEMENTATION AND REPRODUCIBILITY

The code and materials used to generate the datasets, run the automated triage, and perform the agent-based analysis are available at: <https://github.com/MayankKonduri/ate-agent-triage>. Sample PDF reports generated during the triage process are also included in the repository for reference.

————— ◇ —————

REFERENCES

- [1] Advanced Micro Devices, Inc., “Annual report on Form 10-K for the fiscal year ended December 27, 2025,” U.S. Securities and Exchange Commission, 2026. [Online]. Available: <https://www.sec.gov/Archives/edgar/data/2488/000000248826000018/amd-20251227.htm>
- [2] Teradyne, Inc., “Standard test data format (STDF) specification, version 4.” [Online]. Available: STDF V4 Specification.
- [3] Advantest Corporation, “Advantest extends V93000 tester with SmartShell software for tight integration of hardware with Mentor’s Silicon Insight,” May 13, 2019. [Online]. Available: <https://www.advantest.com/en/news/2019/20190513.html>
- [4] A. Khoche, P. Burlison, J. Rowe, and G. Plowman, “A tutorial on STDF fail datalog standard,” in Proc. IEEE Int. Test Conf. (ITC), 2008. [Online]. Available: <https://ieeexplore.ieee.org/document/4700654>
- [5] yieldHUB, “STDF: Why a consistent format for storing test data is essential.” [Online]. Available: <https://www.yieldhub.com/blogs/stdf-why-a-consistent-format-for-storing-test-data-is-essential>
- [6] Automotive Electronics Council, “AEC-Q002 Rev. A: Guidelines for statistical yield analysis,” 2000. [Online]. Available: http://www.aecouncil.com/Documents/AEC_Q002_Rev_A.pdf
- [7] JEDEC Solid State Technology Association, “Customer notification standard for product/process changes by electronic product suppliers (J-STD-046).” [Online]. Available: <https://www.jedec.org/standards-documents/docs/j-std-046>
- [8] Automotive Electronics Council, “AEC-Q100 Rev. J: Failure mechanism based stress test qualification for integrated circuits,” 2023. [Online]. Available: http://www.aecouncil.com/Documents/AEC_Q100_Rev_J_Base_Document.pdf
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learn. Representations (ICLR), 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [10] Y. Qin et al., “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in Proc. Int. Conf. Learn. Representations (ICLR), 2024. [Online]. Available: <https://arxiv.org/abs/2307.16789>
- [11] R. Madge, B. Benware, R. Turakhia, R. Daasch, C. Schuermyer, and J. Ruffler, “In search of the optimum test set: Adaptive test methods for maximum defect coverage and lowest test cost,” in Proc. IEEE Int. Test Conf. (ITC), 2004, pp. 203–212. [Online]. Available: <https://ieeexplore.ieee.org/document/1386954>
- [12] IEEE Electronics Packaging Society, “Heterogeneous integration roadmap: Chapter 17, Test technology,” 2020. [Online]. Available: https://eps.ieee.org/wp-content/uploads/2025/11/ch17_test-2020.pdf
- [13] T. Sakurai and A. R. Newton, “Alpha-power law MOSFET model and its applications to CMOS inverter delay and other formulas,” *IEEE Journal of Solid-State Circuits*, vol. 25, no. 2, pp. 584–594, Apr. 1990, doi: 10.1109/4.52187.
- [14] A. S. Sokolov et al., “Influence of oxygen vacancies in ALD HfO_{2-x} thin films on non-volatile resistive switching,” *Applied Surface Science*, vol. 434, pp. 822–830, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169433217332397>
- [15] Y. Taur and T. H. Ning, *Fundamentals of Modern VLSI Devices*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2009.
- [16] Synopsys, Inc., “Yield Manager datasheet.” [Online]. Available: <https://www.synopsys.com/content/dam/synopsys/silicon/datasheets/yield-manager-ds.pdf>
- [17] DR YIELD GmbH, “Semiconductor test and yield data visualization,” Oct. 2022. [Online]. Available: <https://dryield.com/semiconductor-test-and-yield-data-visualization/>
- [18] R. R. Doering and D. K. de Vries, “Method for analyzing probe yield sensitivities to IC design,” U.S. Patent 6 210 983, Apr. 3, 2001.
- [19] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
- [20] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. R. Stat. Soc., Ser. B*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.
- [21] J. Geng et al., “A survey of confidence estimation and calibration in large language models,” in Proc. NAACL, 2024, pp. 6577–6595.
- [22] E. Peretz-Andersson, N. Haneefa, and T. Lazebnik, “A quality-preserving model for test reduction in electronics production,” *Front. Mech. Eng.*, vol. 12, 2026, Art. no. 1861443, doi: 10.3389/fmech.2026.1861443. [Online]. Available: <https://www.frontiersin.org/journals/mechanical-engineering/articles/10.3389/fmech.2026.1861443/full>